

L E C T U R E N O T E S

CSD622

Pattern Recognition and Data Mining

Master of Science Programme

Tutor: Dr A.J Fofanah

Department of Computer Science

Topics Covered: Feature Extraction · Classification · Clustering · Association Rules Neural Networks · Support Vector Machines · Deep Learning · Evaluation Metrics

Table of Contents

Table of Contents	
Introduction to the Course	5
Learning Objectives	5
How to Use These Notes	5
Module 1: Foundations of Pattern Recognition	5
1.1 The Pattern Recognition Pipeline	5
1.2 Features, Feature Vectors, and Feature Spaces	5
1.2.1 Types of Features	6
1.2.2 The Curse of Dimensionality	6
1.3 Bayesian Decision Theory	7
1.3.1 Bayes' Theorem	7
1.3.2 The Bayes Optimal Classifier	7
1.4 Types of Learning	8
Module 2: Classical Classification Algorithms	9
2.1 k-Nearest Neighbours (k-NN)	9
2.1.1 Algorithm	9
2.1.2 Distance Metrics	9
2.1.3 Choosing k	10
2.2 Naive Bayes Classifier	10
2.2.1 The Naive Bayes Decision Rule	10
2.2.2 Variants by Feature Type	10
2.3 Decision Trees	11
2.3.1 Tree Structure	11
2.3.2 Splitting Criteria	11
2.3.3 Pruning to Prevent Overfitting	12
2.4 Support Vector Machines (SVM)	12
2.4.1 The Maximal Margin Hyperplane	12
2.4.2 Soft Margin SVM	12
2.4.3 The Kernel Trick	13
2.5 Ensemble Methods	13
2.5.1 Bagging (Bootstrap Aggregating)	13
2.5.2 Boosting	13
2.5.3 Stacking	14
Module 3: Neural Networks and Deep Learning	15

3.1 The Perceptron and Multi-Layer Networks.....	15
3.1.1 The Artificial Neuron.....	15
3.1.2 Activation Functions.....	15
3.1.3 Multi-Layer Perceptron (MLP).....	16
3.1.4 The Backpropagation Algorithm.....	16
3.2 Training Deep Neural Networks.....	16
3.2.1 Loss Functions.....	16
3.2.2 Optimisation Algorithms.....	17
3.2.3 Regularisation Techniques.....	17
3.3 Convolutional Neural Networks (CNNs).....	17
3.3.1 Key Layers.....	18
3.3.2 Landmark CNN Architectures.....	18
3.4 Recurrent Neural Networks (RNNs) and LSTMs.....	19
Module 4: Unsupervised Learning — Clustering and Dimensionality Reduction	20
4.1 Clustering.....	20
4.1.1 k-Means Clustering.....	20
4.1.2 Hierarchical Clustering.....	21
4.1.3 DBSCAN — Density-Based Spatial Clustering.....	22
4.2 Dimensionality Reduction.....	22
4.2.1 Principal Component Analysis (PCA).....	22
4.2.2 t-SNE (t-Distributed Stochastic Neighbour Embedding).....	23
4.2.3 UMAP (Uniform Manifold Approximation and Projection).....	23
Module 5: Data Mining — Association Rule Learning	24
5.1 Key Concepts and Metrics.....	24
5.1.1 Support, Confidence, and Lift.....	24
5.2 The Apriori Algorithm.....	25
5.3 FP-Growth Algorithm.....	25
5.4 Evaluating and Interpreting Association Rules.....	26
Module 6: Feature Engineering, Selection, and Extraction	27
6.1 Feature Engineering.....	27
6.1.1 Handling Missing Values.....	27
6.1.2 Encoding Categorical Variables.....	27
6.1.3 Feature Scaling.....	28
6.2 Feature Selection.....	28
6.2.1 Filter Methods.....	28
6.2.2 Wrapper Methods.....	29
6.2.3 Embedded Methods.....	29
6.3 Handling Imbalanced Datasets.....	29

Module 7: Model Evaluation and Validation	31
7.1 Classification Metrics	31
7.1.1 The Confusion Matrix	31
7.1.2 Performance Metrics Derived from the Confusion Matrix	31
7.1.3 ROC Curve and AUC	31
7.2 Validation Strategies	32
7.2.1 Holdout Validation	32
7.2.2 k-Fold Cross-Validation	32
7.3 Bias-Variance Trade-Off	33
7.4 Hyperparameter Tuning	33
7.4.1 Grid Search	34
7.4.2 Random Search	34
7.4.3 Bayesian Optimisation	34
Module 8: Text Mining, Web Mining, and Stream Mining	35
8.1 Text Mining and Natural Language Processing (NLP)	35
8.1.1 Text Pre-processing Pipeline	35
8.1.2 Text Representation	35
8.1.3 Text Classification Tasks	36
8.2 Web Mining	36
8.2.1 PageRank Algorithm	37
8.3 Stream Mining	37
8.3.1 Challenges of Stream Mining	37
8.3.2 Key Algorithms	37
Module 9: Ethics, Fairness, and Responsible Data Mining	39
9.1 Bias in Machine Learning	39
9.1.1 Sources of Bias	39
9.2 Fairness Metrics	39
9.3 Privacy in Data Mining	40
9.4 Explainability and Interpretability	40
Revision Guide and Examination Preparation	42
10.1 Key Concepts Summary by Module	42
Module 1 — Foundations	42
Module 2 — Classical Classification	42
Module 3 — Neural Networks	42
Module 4 — Unsupervised Learning	43
Module 5 — Association Rules	43
Module 6 — Feature Engineering	43

Module 7 — Model Evaluation.....	43
Module 8 — Text, Web, and Stream Mining.....	44
Module 9 — Ethics and Fairness.....	44
10.2 Sample Examination Questions with Guidance.....	44
Question 1 (12 marks).....	44
Question 2 (10 marks).....	45
Question 3 (15 marks).....	45
Question 4 (10 marks).....	46
Question 5 (8 marks).....	46
10.3 Glossary of Key Terms.....	46
10.4 Recommended Reading and Resources.....	48
Primary Textbooks.....	48
Online Resources.....	49
Hands-On Datasets for Practice.....	49

Introduction to the Course

These lecture notes form the primary reference for CSD622: Pattern Recognition and Data Mining, a graduate-level course designed for Master of Science students in Computer Science. The course bridges mathematical theory with practical machine learning, teaching students to build intelligent systems that extract structure, patterns, and knowledge from data.

Pattern recognition is the scientific discipline concerned with the automatic discovery of regularities in data through the use of computer algorithms, and with the use of these regularities to take actions such as classifying data into different categories. Data mining extends this to knowledge discovery in large databases, applying statistical and algorithmic techniques to uncover hidden insights that are non-obvious, previously unknown, and potentially useful.

Learning Objectives

By the end of this course, students will be able to:

- Understand the theoretical foundations of pattern recognition including Bayesian decision theory and statistical learning
- Apply and implement classical classification algorithms: k-NN, Naive Bayes, Decision Trees, SVM, and ensemble methods
- Design and train artificial neural networks and deep learning architectures including CNNs and RNNs
- Perform unsupervised learning: clustering (k-Means, DBSCAN, hierarchical) and dimensionality reduction (PCA, t-SNE)
- Mine association rules and frequent patterns using Apriori and FP-Growth algorithms
- Apply text mining, web mining, and stream mining techniques to real-world datasets
- Rigorously evaluate models using cross-validation, ROC analysis, and statistical hypothesis tests
- Recognise and address ethical issues in pattern recognition and data mining applications

How to Use These Notes

Each module mirrors the lecture slides and expands on every concept with derivations, worked examples, algorithm pseudocode, Python code snippets, and examination guidance. Work through the notes before each lecture, attempt the self-assessment questions, and bring unresolved questions to tutorials.

 **Note:** Mathematics is unavoidable in this course. Do not skip the equations — every formula in these notes is there because it is examined and because understanding it is the difference between applying a method correctly and applying it blindly.

Module 1: Foundations of Pattern Recognition

Before applying any algorithm, we must understand what we are trying to achieve mathematically. This module establishes the formal framework — feature spaces, probability theory, and decision theory — that underpins every method in the course.

1.1 The Pattern Recognition Pipeline

Every pattern recognition system follows the same fundamental pipeline, regardless of the domain (image classification, fraud detection, medical diagnosis, or speech recognition):

1. Data Collection: gather raw observations from the world (pixels, sensor readings, financial transactions)
2. Pre-processing: remove noise, normalise scale, handle missing values, segment objects from background
3. Feature Extraction: transform raw data into a compact, informative numerical representation (the feature vector)
4. Model Selection: choose an algorithm appropriate for the task (supervised, unsupervised, or semi-supervised)
5. Training: fit the model to labelled (or unlabelled) examples to learn decision boundaries or clusters
6. Evaluation: measure performance on held-out test data using appropriate metrics
7. Deployment: embed the model in a production system and monitor for performance degradation

Example — Medical Diagnosis: An automated chest X-ray classifier: (1) collects 200,000 labelled X-rays, (2) resizes to 224×224 pixels and normalises pixel intensities, (3) extracts features using a pre-trained CNN, (4) selects a fine-tuned ResNet-50, (5) trains for 50 epochs with class-weighted loss, (6) evaluates AUC-ROC on 10,000 held-out images, and (7) deploys as a clinical decision support API.

1.2 Features, Feature Vectors, and Feature Spaces

A feature is a measurable property of the phenomenon being observed. A feature vector $x = (x_1, x_2, \dots, x_n)$ is an n -dimensional vector where each component is the value of one feature. The set of all possible feature vectors forms the feature space, also called the input space or pattern space.

Dimensionality: The number of features n in the feature vector. High-dimensional feature spaces ($n > 100$) require special treatment due to the Curse of Dimensionality.

Instance / Sample / Example: A single data point represented as a feature vector, often paired with a label y in supervised learning.

Feature Matrix X : An $m \times n$ matrix where each of the m rows is a feature vector (instance) and each of the n columns is a feature.

1.2.1 Types of Features

Type	Description	Example	Encoding
Continuous / Ratio	Real-valued measurements with a natural zero	Height: 1.75m, Temperature: 37.2°C	Use directly after normalisation
Ordinal	Ordered categories without equal intervals	Education level: primary < secondary < degree	Integer encoding (1, 2, 3)
Nominal / Categorical	Unordered categories	Blood type: A, B, AB, O	One-hot encoding
Binary	Two possible values	Smoker: yes/no	0 or 1
Text	Sequences of words	Customer review text	TF-IDF, word embeddings
Image	Pixel arrays	64×64 greyscale face image	Pixel values, CNN features

1.2.2 The Curse of Dimensionality

As the number of features n increases, the volume of the feature space grows exponentially. This creates three compounding problems:

- Sparsity: for a fixed number of training samples m , the samples become increasingly sparse as n grows — the 'nearest neighbours' become far away and may not be representative
- Distance concentration: in high dimensions, all pairwise distances tend to become similar, making distance-based methods (k-NN, k-Means) unreliable
- Overfitting risk: with many features and few samples, a model can memorise the training data perfectly while failing to generalise

Required samples $\propto 2^n$ (exponential growth with dimensionality n)

 **Note:** Practical rule of thumb: you need at least 5–10 training samples per feature for a linear model, and exponentially more for non-linear models. Always reduce dimensionality before applying high-complexity algorithms to small datasets.

1.3 Bayesian Decision Theory

Bayesian decision theory provides the optimal theoretical framework for pattern classification. It asks: given the feature vector x , what is the probability that the class label y belongs to each class c ? The answer is given by Bayes' theorem.

1.3.1 Bayes' Theorem

$$P(c \mid x) = P(x \mid c) \cdot P(c) / P(x)$$

Where:

- $P(c \mid x)$ is the posterior probability — the probability of class c given the observed feature vector x . This is what we want.
- $P(x \mid c)$ is the class-conditional likelihood — the probability of observing x given that the true class is c .
- $P(c)$ is the prior probability — our prior belief about how common class c is in the population.
- $P(x)$ is the marginal likelihood (evidence) — a normalisation constant ensuring probabilities sum to 1.

1.3.2 The Bayes Optimal Classifier

The Bayes optimal decision rule assigns x to the class with the highest posterior probability:

$$\hat{c} = \operatorname{argmax}_c P(c \mid x) = \operatorname{argmax}_c [P(x \mid c) \cdot P(c)]$$

This is the Bayes optimal classifier — no other classifier can achieve a lower error rate when the true class-conditional densities $P(x \mid c)$ are known. The error rate of the Bayes classifier is called the Bayes error rate, and it is a lower bound on any classifier's error rate.

Example — Spam Detection: Suppose $P(\text{spam}) = 0.3$ and $P(\text{not spam}) = 0.7$. For a given email feature vector x , suppose $P(x \mid \text{spam}) = 0.8$ and $P(x \mid \text{not spam}) = 0.1$. Then: $P(\text{spam} \mid x) \propto 0.8 \times 0.3 = 0.24$, and $P(\text{not spam} \mid x) \propto 0.1 \times 0.7 = 0.07$. Since $0.24 > 0.07$, the email is classified as spam. Note: $P(x)$ cancels in the comparison so we do not need to compute it.

1.4 Types of Learning

Learning Type	Training Data	Goal	Examples
Supervised	Labelled pairs (x, y)	Learn mapping f: $x \rightarrow y$	Classification, regression
Unsupervised	Unlabelled data {x}	Discover structure	Clustering, PCA, generative models
Semi-supervised	Few labels + many unlabelled	Leverage unlabelled data	Label propagation, self-training
Reinforcement	Reward signal from environment	Learn optimal policy	Game playing, robotics, RL agents
Self-supervised	Unlabelled data with pretext tasks	Pre-train representations	BERT, GPT, SimCLR, BYOL

Module 2: Classical Classification Algorithms

This module covers the foundational classification algorithms that remain widely used in practice. Understanding these methods — their assumptions, strengths, and weaknesses — is essential for selecting the right tool for each task and for understanding why modern deep learning methods work.

2.1 k-Nearest Neighbours (k-NN)

k-NN is a non-parametric, instance-based (lazy) learning algorithm. It makes no assumptions about the functional form of the decision boundary. Classification is based on the labels of the k training examples nearest to the query point in feature space.

2.1.1 Algorithm

8. Given a query point x_q and a labelled training set $\{(x_1, y_1), \dots, (x_m, y_m)\}$
9. Compute the distance $d(x_q, x_i)$ between x_q and every training point x_i
10. Select the k training points with smallest distance — the k -nearest neighbours $N_k(x_q)$
11. Assign x_q to the class that appears most frequently among its k nearest neighbours (majority vote)

$$\hat{c} = \operatorname{argmax}_c \sum_{i \in N_k(x_q)} 1[y_i = c]$$

2.1.2 Distance Metrics

Metric	Formula	Best Used For
Euclidean	$d = \sqrt{\sum (x_i - y_i)^2}$	Continuous features, roughly equal scale
Manhattan (L1)	$d = \sum x_i - y_i $	High-dimensional sparse data, robust to outliers
Minkowski	$d = (\sum x_i - y_i ^p)^{1/p}$	Generalisation: $p=1$ Manhattan, $p=2$ Euclidean
Cosine	$d = 1 - (x \cdot y) / (x y)$	Text/document similarity (direction matters, not magnitude)
Hamming	$d = (1/n) \sum 1[x_i \neq y_i]$	Categorical / binary features

2.1.3 Choosing k

- $k = 1$: lowest bias but highest variance — sensitive to noise and outliers (overfitting)
- Large k : smoother decision boundary, lower variance, but may blur class boundaries (underfitting)
- Rule of thumb: start with $k = \sqrt{m}$ where m is the number of training samples; tune via cross-validation
- Use odd k for binary classification to avoid ties

Example — k-NN Image Classification: In the MNIST handwritten digit dataset (70,000 28×28 images), k-NN with $k=3$ and Euclidean distance achieves about 97% accuracy. The 784-dimensional feature vectors (pixel values) are compared directly. While accurate, prediction requires comparing every test image to all 70,000 training images — $O(m \cdot n)$ time per query — making it slow at scale.

2.2 Naive Bayes Classifier

Naive Bayes applies Bayes' theorem with the 'naive' assumption that features are conditionally independent given the class. Despite this assumption almost never being true in practice, Naive Bayes works surprisingly well across many domains and is extremely fast to train and predict.

2.2.1 The Naive Bayes Decision Rule

$$P(c | x) \propto P(c) \cdot \prod_{j=1}^n P(x_j | c)$$

Taking the log to avoid numerical underflow (products of small probabilities):

$$\log P(c | x) \propto \log P(c) + \sum_{j=1}^n \log P(x_j | c)$$

2.2.2 Variants by Feature Type

- Gaussian Naive Bayes: assumes $P(x_j | c) \sim N(\mu_{jc}, \sigma_{jc}^2)$. Used for continuous features. Estimates μ and σ^2 from training data per class.
- Multinomial Naive Bayes: assumes features are counts (e.g., word frequencies). The dominant choice for text classification.
- Bernoulli Naive Bayes: assumes binary features (e.g., word presence/absence). Suitable for short texts.

- Complement Naive Bayes: trains each class classifier on all other classes' data. More robust for imbalanced text datasets.

📌 **Note:** Laplace smoothing (additive smoothing) adds a small constant α (typically 1) to all feature counts to prevent zero probabilities when a feature value never appeared with a class in training. Without it, a single unseen feature makes the entire posterior zero.

2.3 Decision Trees

Decision trees are hierarchical models that partition the feature space into axis-aligned regions. They are intuitive, interpretable, and handle mixed feature types (continuous and categorical) without preprocessing.

2.3.1 Tree Structure

- Root node: the first split, applied to the entire training set
- Internal node: a decision based on one feature (e.g., age < 35 or colour = 'red')
- Branch: the outcome of a decision (left for True/≤, right for False/>)
- Leaf node: a terminal node that outputs a class label (classification) or numerical value (regression)

2.3.2 Splitting Criteria

To select the best feature to split on at each node, we measure the reduction in impurity. The most common measures are:

Entropy (Shannon): $H(S) = - \sum p_i \log_2 p_i$ (measures disorder; 0 = pure, 1 = maximum disorder for binary classes)

Information Gain: $IG(S, A) = H(S) - \sum_v (|S_v|/|S|) \cdot H(S_v)$ (used by ID3/C4.5 — the drop in entropy after splitting on attribute A)

Gini Impurity: $Gini(S) = 1 - \sum p_i^2$ (used by CART — equivalent to probability of misclassifying a randomly chosen element)

Example — Gini Split Calculation: A node has 30 positive and 70 negative samples (Gini = $1 - (0.3^2 + 0.7^2) = 0.42$). Splitting on feature A creates left child (25 positive, 5 negative; Gini = 0.278) and right child (5 positive, 65 negative; Gini = 0.132). Weighted Gini after split = $(30/100) \times 0.278 + (70/100) \times 0.132 = 0.176$. Gini reduction = $0.42 - 0.176 = 0.244$.

2.3.3 Pruning to Prevent Overfitting

- Pre-pruning (early stopping): stop splitting when the gain falls below a threshold, the node has fewer than `min_samples_split` examples, or the tree reaches `max_depth`
- Post-pruning (reduced error pruning): grow the full tree, then remove branches that do not improve accuracy on a validation set
- Cost-complexity pruning (CART): minimise $\alpha \cdot |T| + \text{error}(T)$ where $|T|$ is the number of leaves and α is a regularisation parameter tuned by cross-validation

2.4 Support Vector Machines (SVM)

SVMs find the hyperplane that maximally separates classes in feature space. They are one of the most theoretically principled classification methods and were the dominant approach before deep learning became practical.

2.4.1 The Maximal Margin Hyperplane

For a linearly separable binary classification problem with labels $y \in \{-1, +1\}$, an SVM seeks the hyperplane $w \cdot x + b = 0$ that maximises the margin — the distance between the hyperplane and the nearest training points of each class (the support vectors).

$$\text{Maximise } 2/||w|| \quad \text{subject to} \quad y_i(w \cdot x_i + b) \geq 1 \quad \forall i$$

Equivalently, this is a quadratic programming problem: minimise $(1/2)||w||^2$ subject to the constraints. The decision function classifies a new point x as:

$$f(x) = \text{sign}(w \cdot x + b)$$

2.4.2 Soft Margin SVM

When the data is not linearly separable, we allow some misclassification by introducing slack variables $\xi_i \geq 0$:

$$\text{Minimise } (1/2)||w||^2 + C \cdot \sum \xi_i \quad \text{subject to} \quad y_i(w \cdot x_i + b) \geq 1 - \xi_i$$

The hyperparameter C controls the bias-variance trade-off: small $C \rightarrow$ wide margin, more misclassifications tolerated (high bias, low variance); large $C \rightarrow$ narrow margin, fewer misclassifications (low bias, high variance).

2.4.3 The Kernel Trick

SVMs can classify non-linearly separable data by implicitly mapping features into a higher-dimensional space using a kernel function $K(x_i, x_j) = \phi(x_i) \cdot \phi(x_j)$. The kernel trick avoids computing $\phi(x)$ explicitly — it only requires kernel evaluations.

Kernel	Formula	Use Case
Linear	$K(x, z) = x \cdot z$	Text classification, high-dimensional sparse features
Polynomial	$K(x, z) = (yx \cdot z + r)^d$	Image features, interactions up to degree d
RBF (Gaussian)	$K(x, z) = \exp(-\gamma \ x - z\ ^2)$	General-purpose; most commonly used default
Sigmoid	$K(x, z) = \tanh(yx \cdot z + r)$	Neural network-like; less commonly used

2.5 Ensemble Methods

Ensemble methods combine multiple base learners to produce a stronger predictor than any individual model. They are among the most effective practical approaches, consistently winning competitions and outperforming single models.

2.5.1 Bagging (Bootstrap Aggregating)

Bagging creates B training sets by sampling with replacement (bootstrapping) from the original training data. A separate model is trained on each bootstrap sample. Predictions are combined by majority vote (classification) or averaging (regression). Bagging reduces variance without increasing bias.

Random Forest: An extension of bagging specifically for decision trees: each tree also randomly selects a subset of $\sqrt{n}$ features to consider at each split. This de-correlates the trees, reducing variance further. Typically uses 100–500 trees.

2.5.2 Boosting

Boosting builds an additive ensemble sequentially: each new model focuses on the examples that previous models got wrong. The final prediction is a weighted combination of all models. Boosting reduces bias (and can also reduce variance).

AdaBoost: Fits successive weak classifiers (typically decision stumps) to re-weighted training data. Misclassified examples get higher weights. Final classifier is a weighted majority vote.

Gradient Boosting: Fits each new tree to the residuals (pseudo-residuals from the loss gradient) of the current ensemble. Implementations: XGBoost, LightGBM, CatBoost — among the most powerful algorithms for tabular data.

2.5.3 Stacking

Stacking trains a meta-learner on the out-of-fold predictions of base models. The meta-learner learns how to best combine the base models' outputs. Typically uses 3–5 diverse base models (e.g., RF + SVM + k-NN) and a simple meta-learner (logistic regression).

Method	Combines How	Reduces	Risk	Examples
Bagging	Parallel (bootstrap samples)	Variance	Slight increase in bias	Random Forest, Extra Trees
Boosting	Sequential (re-weighted)	Bias	Overfitting if too many rounds	AdaBoost, XGBoost, LightGBM
Stacking	Meta-learner on predictions	Both	Complexity, data leakage	Multi-level stacking ensembles

Module 3: Neural Networks and Deep Learning

Artificial neural networks are computational models inspired by biological neural systems. Deep learning — the training of neural networks with many layers — has revolutionised pattern recognition, achieving human-level or superhuman performance on image recognition, speech recognition, natural language processing, and game playing.

3.1 The Perceptron and Multi-Layer Networks

3.1.1 The Artificial Neuron

An artificial neuron receives a vector of inputs $x = (x_1, \dots, x_n)$, computes a weighted sum, adds a bias, and passes the result through a non-linear activation function:

$$z = w_1x_1 + w_2x_2 + \dots + w_nx_n + b = w \cdot x + b$$

$$a = f(z) \quad \text{where } f \text{ is the activation function}$$

3.1.2 Activation Functions

Function	Formula	Range	Use / Notes
Sigmoid	$\sigma(z) = 1/(1 + e^{-z})$	(0, 1)	Output layer for binary classification; suffers from vanishing gradient in deep nets
Tanh	$\tanh(z) = (e^z - e^{-z}) / (e^z + e^{-z})$	(-1, 1)	Zero-centred, better than sigmoid for hidden layers; still vanishing gradient
ReLU	$\text{ReLU}(z) = \max(0, z)$	$[0, \infty)$	Default for hidden layers; fast, avoids vanishing gradient; can suffer 'dying ReLU'
Leaky ReLU	$\max(\alpha z, z) \quad \alpha \approx 0.01$	$(-\infty, \infty)$	Fixes dying ReLU by allowing small negative gradient
Softmax	$\text{softmax}(z_i) = e^{z_i} / \sum e^{z_j}$	(0,1), sums to 1	Output layer for multi-class classification; outputs class probabilities
GELU	$z \cdot \Phi(z)$	$(-\infty, \infty)$	Used in Transformer models (BERT, GPT);

			smooth approximation of ReLU
--	--	--	------------------------------

3.1.3 Multi-Layer Perceptron (MLP)

A Multi-Layer Perceptron (also called a feedforward neural network or fully-connected network) consists of an input layer, one or more hidden layers, and an output layer. The universal approximation theorem states that a single hidden layer with enough neurons can approximate any continuous function — though in practice deep networks (multiple layers) are far more parameter-efficient.

3.1.4 The Backpropagation Algorithm

Backpropagation is the algorithm for computing the gradient of the loss function with respect to every weight in the network, using the chain rule of calculus. The gradients are then used to update weights via gradient descent.

12. Forward pass: compute activations layer by layer from input to output; compute the loss L
13. Backward pass: compute $\partial L / \partial w$ for each weight w by propagating gradients backward from the output layer to the input layer
14. Weight update: $w \leftarrow w - \eta \cdot \partial L / \partial w$ where η is the learning rate

$$\partial L / \partial w_{ij}^{(l)} = \partial L / \partial a_j^{(l)} \cdot f'(z_j^{(l)}) \cdot a_i^{(l-1)} \quad (\text{chain rule})$$

3.2 Training Deep Neural Networks

3.2.1 Loss Functions

Task	Loss Function	Formula
Binary classification	Binary Cross-Entropy	$L = -[y \log(\hat{y}) + (1-y) \log(1-\hat{y})]$
Multi-class classification	Categorical Cross-Entropy	$L = -\sum y_i \log(\hat{y}_i)$
Regression	Mean Squared Error (MSE)	$L = (1/m) \sum (y_i - \hat{y}_i)^2$
Regression (robust)	Mean Absolute Error	$L = (1/m) \sum y_i - \hat{y}_i $

	(MAE)	
Margin-based	Hinge Loss	$L = \max(0, 1 - yf(x))$ (SVM)

3.2.2 Optimisation Algorithms

Stochastic Gradient Descent (SGD): Updates weights using the gradient computed on a single randomly chosen training example (or small mini-batch). Noisy but fast; can escape local minima.

Momentum: Accumulates an exponentially decaying moving average of past gradients to accelerate SGD in consistent directions and dampen oscillations. $v \leftarrow \beta v - \eta \nabla L$; $w \leftarrow w + v$

Adam (Adaptive Moment Estimation): Combines momentum (first moment) and RMSProp (second moment) to adapt the learning rate per parameter. The default optimiser for most deep learning applications.

3.2.3 Regularisation Techniques

- L2 regularisation (Weight Decay): adds $\lambda \|w\|^2$ to the loss, penalising large weights and encouraging smoother models
- Dropout: randomly sets a fraction p of neuron activations to zero during each training step. Forces the network to learn redundant representations. Equivalent to training an ensemble of 2^n sub-networks.
- Batch Normalisation: normalises the pre-activation values within each mini-batch to zero mean and unit variance, then applies learnable scale γ and shift β . Stabilises training, enables higher learning rates.
- Early Stopping: monitor validation loss during training; stop when it stops decreasing (before the training loss gap to validation loss grows too large)

3.3 Convolutional Neural Networks (CNNs)

CNNs exploit the spatial structure of images by applying locally-connected, weight-shared filters (convolutions). They are the dominant architecture for image classification, object detection, and segmentation.

3.3.1 Key Layers

Convolutional Layer: Applies a bank of learnable filters (kernels) of size $k \times k$ across the input. Each filter detects a specific local pattern (edge, texture, colour blob). Produces a feature map. Parameters: kernel size, stride, padding.

Pooling Layer: Reduces spatial dimensions by summarising a region with max (Max Pooling) or average (Average Pooling). Provides translational invariance and reduces computational cost.

Fully-Connected Layer: Standard MLP layers applied after flattening the feature maps. Typically only the final 1–2 layers; handles high-level reasoning.

3.3.2 Landmark CNN Architectures

Architecture	Year	Innovation	ImageNet Top-5 Error
LeNet-5	1998	First successful deep CNN for digit recognition	N/A (MNIST)
AlexNet	2012	ReLU, Dropout, GPU training — started the deep learning era	15.3%
VGGNet	2014	Deep, uniform 3×3 convolutions, simple and effective	7.3%
GoogLeNet (Inception)	2014	Inception modules: parallel branches of different kernel sizes	6.7%
ResNet	2015	Residual (skip) connections enabling 100+ layers without vanishing gradient	3.57%
EfficientNet	2019	Compound scaling of depth, width, and resolution	1.8%

Example — Transfer Learning: Training a CNN from scratch for medical imaging requires millions of labelled examples. Instead, take a ResNet-50 pre-trained on ImageNet (1.2M images, 1,000 classes), replace the final classification layer with a new layer matching the number of medical classes, and fine-tune the entire network with a small learning rate on the medical dataset (even with only 1,000 labelled images). This typically achieves 90%+ of the performance of full training.

3.4 Recurrent Neural Networks (RNNs) and LSTMs

RNNs process sequential data by maintaining a hidden state that captures information from previous time steps. They are used for time series, speech, text, and any domain where temporal ordering matters.

$$\mathbf{h}_t = \mathbf{f}(\mathbf{W}_h \cdot \mathbf{h}_{t-1} + \mathbf{W}_x \cdot \mathbf{x}_t + \mathbf{b})$$

The fundamental problem with vanilla RNNs is the vanishing gradient: gradients decay exponentially as they are backpropagated through time, making it impossible to learn long-range dependencies.

LSTM (Long Short-Term Memory): Introduces a cell state C_t and three gates: forget gate (what to erase from cell state), input gate (what new information to store), and output gate (what to output). The cell state allows gradients to flow unchanged over long sequences.

GRU (Gated Recurrent Unit): A simpler variant of LSTM with two gates (reset and update). Fewer parameters than LSTM; often comparable performance on many tasks.

 **Note:** For most modern NLP tasks, Transformer-based models (BERT, GPT, T5) have largely superseded RNNs/LSTMs due to their ability to capture long-range dependencies in parallel, without sequential computation.

Module 4: Unsupervised Learning — Clustering and Dimensionality Reduction

Unsupervised learning finds structure in data without labels. Clustering groups similar examples together; dimensionality reduction finds compact representations. Both are essential when labelled data is scarce, expensive, or simply unavailable.

4.1 Clustering

Clustering partitions a dataset into groups (clusters) such that instances within a cluster are more similar to each other than to instances in other clusters. The definition of 'similar' — encoded in a distance or similarity measure — fundamentally shapes the clusters found.

4.1.1 k-Means Clustering

k-Means is the most widely used clustering algorithm. It partitions n instances into k clusters by alternately assigning each instance to the nearest centroid and recomputing centroids.

Algorithm (Lloyd's Algorithm):

15. Initialise k cluster centroids $\mu_1, \dots, \mu_k$ (randomly, or with k-Means++ for better initialisation)
16. Assignment step: assign each instance x_i to the cluster with the nearest centroid: $c_i = \operatorname{argmin}_k ||x_i - \mu_k||^2$
17. Update step: recompute each centroid as the mean of all instances assigned to it: $\mu_k = (1/|C_k|) \sum_{i \in C_k} x_i$
18. Repeat steps 2–3 until convergence (assignments stop changing)

Objective: Minimise WCSS $= \sum_k \sum_{i \in C_k} ||x_i - \mu_k||^2$

k-Means minimises the Within-Cluster Sum of Squares (WCSS), also called inertia. It converges to a local optimum; running multiple random restarts and keeping the best solution is standard practice.

Choosing k — The Elbow Method:

Plot WCSS against k . As k increases, WCSS decreases. The 'elbow' — where the rate of decrease sharply flattens — suggests the optimal k . The silhouette score provides a more principled measure:

$$\text{Silhouette}(i) = (b(i) - a(i)) / \max(a(i), b(i))$$

Where $a(i)$ is the mean intra-cluster distance and $b(i)$ is the mean distance to the nearest other cluster. Ranges from -1 (wrong cluster) to $+1$ (well-clustered).

 **Note:** k-Means assumes clusters are spherical, equally-sized, and well-separated. It fails on non-convex clusters (use DBSCAN), clusters of very different densities, and high-dimensional data (use spectral clustering or dimensionality reduction first).

4.1.2 Hierarchical Clustering

Hierarchical clustering builds a tree of clusters (dendrogram) by either merging (agglomerative, bottom-up) or splitting (divisive, top-down) clusters. The dendrogram can be cut at any height to produce any desired number of clusters.

Agglomerative Clustering Algorithm:

19. Start with each instance as its own cluster (n clusters)
20. Compute pairwise distances between all clusters
21. Merge the two closest clusters into one
22. Update the distance matrix (using the chosen linkage criterion)
23. Repeat steps 3–4 until only one cluster remains

Linkage	Distance Between Clusters A, B	Tendency
Single	min distance between any point in A and any in B	Elongated, 'chaining' clusters
Complete	max distance between any point in A and any in B	Compact, spherical clusters
Average (UPGMA)	Mean distance between all pairs in $A \times B$	Balanced; good general choice
Ward	Increase in WCSS when merging A and B	Compact, equal-size clusters; most popular

4.1.3 DBSCAN — Density-Based Spatial Clustering

DBSCAN (Density-Based Spatial Clustering of Applications with Noise) finds clusters of arbitrary shape by connecting high-density regions. It does not require specifying k in advance and naturally identifies noise points (outliers).

Key Concepts:

ϵ (epsilon): Neighbourhood radius — the maximum distance between two points for them to be considered neighbours

MinPts: Minimum number of points required in the ϵ -neighbourhood of a point for it to be a core point

Core Point: A point with at least MinPts points within distance ϵ (including itself)

Border Point: A non-core point within ϵ of a core point

Noise Point: Neither core nor border — an outlier

Clusters are formed by connecting all density-reachable core points. DBSCAN's time complexity is $O(n \log n)$ with spatial indexing, making it practical for large datasets. Parameters ϵ and MinPts must be chosen carefully — too small ϵ and everything is noise; too large and distinct clusters merge.

Example — DBSCAN for Geospatial Data: To cluster GPS locations of taxi pickups in a city, DBSCAN with $\epsilon=50\text{m}$ and $\text{MinPts}=10$ naturally finds dense pickup hotspots (airports, train stations, restaurants) as clusters, and isolated locations as noise — without needing to specify the number of clusters in advance.

4.2 Dimensionality Reduction

Dimensionality reduction compresses high-dimensional data into a lower-dimensional representation while preserving as much relevant structure as possible. It serves three purposes: visualisation (2D/3D), noise removal, and addressing the Curse of Dimensionality.

4.2.1 Principal Component Analysis (PCA)

PCA finds the orthogonal directions of maximum variance in the data (principal components) and projects data onto the top- k of these directions. It is a linear transformation.

PCA Algorithm:

24. Centre the data: subtract the mean from each feature ($X_{\text{centred}} = X - \mu$)
25. Compute the covariance matrix: $\Sigma = (1/m) X_{\text{centred}}^T X_{\text{centred}}$

26. Compute eigendecomposition: $\Sigma = V \Lambda V^T$ (V: eigenvectors = principal components, Λ : eigenvalues = variances)
27. Sort eigenvectors by descending eigenvalue
28. Project data onto top-k eigenvectors: $Z = X_{\text{centred}} \cdot V_k$

$$\text{Explained Variance Ratio}_k = \lambda_k / \sum_j \lambda_j$$

Choose k to retain a target fraction of variance (commonly 95% or 99%). Plot the cumulative explained variance ratio against k (scree plot) to decide.

Example — PCA for Face Recognition: The 'Eigenfaces' method applies PCA to a dataset of 64×64 greyscale face images (4,096 dimensions). The top 100 principal components (eigenfaces) capture ~95% of the variance. A new face image is projected onto these 100 components, yielding a 100-dimensional feature vector — a 40× dimensionality reduction — that is then classified with k-NN.

4.2.2 t-SNE (t-Distributed Stochastic Neighbour Embedding)

t-SNE is a non-linear dimensionality reduction technique designed specifically for visualisation in 2D or 3D. It preserves local neighbourhood structure — points that are close in high-dimensional space remain close in the low-dimensional embedding.

t-SNE models the pairwise similarities between high-dimensional points as probabilities using a Gaussian distribution, and between low-dimensional embeddings using a heavier-tailed Student-t distribution. It minimises the KL-divergence between these two distributions via gradient descent.

 **Note:** t-SNE is not suitable for general dimensionality reduction before classification (it is non-parametric, so new points cannot be projected). It is a visualisation tool. The axes of a t-SNE plot have no interpretable meaning; only cluster separation and local proximity are meaningful.

4.2.3 UMAP (Uniform Manifold Approximation and Projection)

UMAP is a newer non-linear dimensionality reduction method that is faster than t-SNE, scales to millions of points, and better preserves global structure. It has largely replaced t-SNE in modern practice and can also be used as a preprocessing step before classification.

Module 5: Data Mining — Association Rule Learning

Association rule mining discovers interesting relationships between variables in large transactional databases. Originally developed for market basket analysis, it is now applied to web mining, medical diagnosis, network intrusion detection, and recommendation systems.

5.1 Key Concepts and Metrics

Itemset: A set of one or more items, e.g., {bread, milk, butter}. A k-itemset contains k items.

Transaction: A record in the database representing a set of items purchased or observed together, e.g., $T_1 = \{\text{bread, milk, eggs}\}$.

Frequent Itemset: An itemset whose support meets a minimum threshold (minsup).

5.1.1 Support, Confidence, and Lift

$$\text{Support}(X) = |\text{transactions containing } X| / |\text{total transactions}|$$

$$\text{Support}(X \rightarrow Y) = \text{Support}(X \cup Y)$$

$$\text{Confidence}(X \rightarrow Y) = \text{Support}(X \cup Y) / \text{Support}(X)$$

$$\text{Lift}(X \rightarrow Y) = \text{Confidence}(X \rightarrow Y) / \text{Support}(Y)$$

Metric	Interpretation	Value Range	Rule Quality
Support	How frequently the rule's items appear together	[0, 1]	High support = common pattern
Confidence	How often the rule is correct (precision of the rule)	[0, 1]	High confidence = reliable rule
Lift	How much more likely Y is given X, vs. by chance	[0, ∞)	Lift > 1 = positive correlation; = 1 = independent; < 1 = negative

Example — Market Basket Analysis: In 1,000 transactions, bread and butter are bought together 200 times (Support = 0.2). Bread alone appears in 300 transactions. Confidence(bread → butter) = 200/300 = 0.67. Butter alone appears in 400 transactions, so Support(butter) = 0.4. Lift = 0.67/0.4 = 1.67. This means buying bread makes customers 67% more likely to also buy butter than random chance would suggest.

5.2 The Apriori Algorithm

Apriori exploits the antimonotone (downward closure) property of support: if an itemset is infrequent, all its supersets are also infrequent. This allows pruning the exponential search space.

Antimonotone Property: if $\text{Support}(X) < \text{minsup}$, then $\text{Support}(X \cup Y) < \text{minsup}$ for any Y

Apriori Algorithm:

29. Find all frequent 1-itemsets (L_1) by scanning the database and counting individual item frequencies
30. Generate candidate k -itemsets (C_k) by joining frequent $(k-1)$ -itemsets with themselves (if they share $k-2$ items)
31. Prune candidates: remove any C_k itemset that has an infrequent $(k-1)$ -item subset
32. Scan database to count support of remaining candidates; keep those meeting $\text{minsup} \rightarrow L_k$
33. Repeat steps 2–4 until no new frequent itemsets are found
34. Generate association rules from all frequent itemsets: for each subset $X \subset L$, emit rule $X \rightarrow (LX)$ if confidence $\geq \text{minconf}$

Apriori's main drawback is the need for multiple database scans (one per itemset size) and the potentially huge number of candidate itemsets. It is practical for sparse, low-dimensional datasets but slow for dense, high-dimensional ones.

5.3 FP-Growth Algorithm

FP-Growth (Frequent Pattern Growth) avoids repeated database scans and candidate generation by compressing the database into an FP-tree — a compact prefix-tree structure. Mining proceeds by recursively constructing conditional FP-trees for each item.

FP-Growth Steps:

35. Scan database once to find all frequent 1-itemsets; sort by descending support frequency
36. Scan database again to build the FP-tree: insert each transaction (items ordered by descending frequency) as a path; share prefixes

37. For each frequent item i (in ascending support order), extract the conditional pattern base (all paths leading to i in the FP-tree), build the conditional FP-tree, and recursively mine it

Example — FP-Growth Speed Advantage: On a dataset with 10,000 transactions and 500 items, Apriori requires approximately 20 database scans and generates millions of candidate itemsets. FP-Growth requires exactly 2 database scans and mines directly from the compressed in-memory FP-tree — typically 10–100× faster.

5.4 Evaluating and Interpreting Association Rules

Support and confidence alone are insufficient for rule quality. Several additional measures help filter spurious or uninteresting rules:

- Lift > 1 : positive correlation between X and Y (more useful than chance)
- Conviction: measures directional implication strength — high conviction means X strongly implies Y
- Leverage: $\text{Support}(X \cup Y) - \text{Support}(X) \cdot \text{Support}(Y)$. Positive means X and Y occur together more than expected under independence.
- Zhang's metric, Kulc, Cosine similarity: more sophisticated symmetrical interestingness measures robust to skewed support distributions

 **Note:** In a real retail dataset with thousands of items, mining with low minsup generates millions of rules. Always apply domain knowledge to filter: rules involving only one item, trivially obvious rules (shampoo $\rightarrow$ conditioner), or rules with very low lift should be discarded before presenting results to business stakeholders.

Module 6: Feature Engineering, Selection, and Extraction

'Data preparation and feature engineering take 80% of a data scientist's time.' — common industry axiom. Raw data is rarely in the form that machine learning algorithms require or prefer. This module covers the craft of transforming data into features that maximally inform the learning algorithm.

6.1 Feature Engineering

6.1.1 Handling Missing Values

- Deletion: remove rows (listwise) or columns with missing values — only safe when missingness is rare and random (MCAR)
- Mean/Median/Mode Imputation: replace missing values with the column's mean (continuous), median (skewed/outliers), or mode (categorical). Simple but can reduce variance and distort correlations.
- Model-based Imputation: use k-NN imputation (impute from k nearest complete neighbours) or regression imputation (predict missing feature from other features)
- Multiple Imputation (MICE): create multiple imputed datasets, fit models on each, and pool results. Gold standard for missing data but computationally expensive.
- Indicator variable: add a binary 'is_missing' feature alongside the imputed value — allows the model to learn from the fact that data was missing

6.1.2 Encoding Categorical Variables

- One-hot encoding: create a binary column for each category. Produces $k-1$ new columns for a feature with k categories (dropping one to avoid the dummy variable trap). Suitable for nominal features with few categories.
- Ordinal encoding: integer 1, 2, 3, ... for ordered categories. Preserves order but implies equal spacing.
- Target encoding (mean encoding): replace each category with the mean of the target variable for that category. Powerful but requires careful regularisation to prevent data leakage.
- Embeddings: learn a dense low-dimensional vector representation for each category. Used for high-cardinality categoricals (zip codes, user IDs) in neural networks.

6.1.3 Feature Scaling

Distance-based algorithms (k-NN, SVM, k-Means) and gradient-based optimisation are sensitive to feature scale. Always scale before applying these methods.

Method	Formula	Output Range	Best For
Min-Max Normalisation	$x' = (x - \min) / (\max - \min)$	[0, 1]	Bounded features; neural network inputs
Z-score Standardisation	$x' = (x - \mu) / \sigma$	Mean=0, Std=1	Algorithms assuming Gaussian features; SVM, PCA
Robust Scaler	$x' = (x - Q_2) / (Q_3 - Q_1)$	Varies	Data with outliers (uses median and IQR)
Log Transform	$x' = \log(x + 1)$	Varies	Right-skewed features (income, population)

6.2 Feature Selection

Feature selection identifies the most relevant features for a given task, reducing dimensionality, removing noise, and improving both model accuracy and interpretability.

6.2.1 Filter Methods

Filter methods score features independently of the model using statistical tests. They are fast but do not account for feature interactions.

- Variance Threshold: remove features with near-zero variance (constant or near-constant features carry no information)
- Pearson Correlation: remove features highly correlated with other features (multicollinearity) or select features highly correlated with the target
- Mutual Information: measures how much knowing one feature reduces uncertainty about the target. Captures non-linear dependencies.
- Chi-squared test: for categorical features vs. categorical targets — tests independence between each feature and the class label
- ANOVA F-test: for continuous features vs. categorical targets — tests whether the feature means differ significantly across classes

6.2.2 Wrapper Methods

Wrapper methods evaluate subsets of features by training and evaluating a model. More accurate than filters but computationally expensive.

- Forward Selection: start with no features; greedily add the feature that most improves model performance; stop when adding more features does not help
- Backward Elimination: start with all features; greedily remove the feature whose removal least harms performance
- Recursive Feature Elimination (RFE): train the model, remove the least important feature (by weight magnitude or importance score), repeat

6.2.3 Embedded Methods

Embedded methods perform feature selection as part of model training.

- L1 (Lasso) regularisation: adds $\lambda \sum |w_j|$ to the loss. Drives irrelevant feature weights exactly to zero — automatic sparse feature selection.
- Random Forest Feature Importance: the mean decrease in Gini impurity (or permutation importance) across all trees. A fast, reliable embedded feature selector.
- XGBoost Feature Importance: similar to Random Forest but also provides gain-based and SHAP-based importance scores

6.3 Handling Imbalanced Datasets

In many practical classification problems — fraud detection, medical diagnosis, rare event prediction — one class is vastly rarer than others. A model that predicts 'not fraud' for everything achieves 99.9% accuracy on a dataset where only 0.1% of transactions are fraudulent, but is completely useless.

Technique	How It Works	When to Use
Under-sampling (Random)	Randomly remove majority class samples	Very large dataset; information can be spared
Over-sampling (Random)	Randomly duplicate minority class samples	Small dataset; risk of overfitting on duplicates
SMOTE	Generate synthetic minority samples by interpolating between existing ones	Standard choice for moderate imbalance

Class weights	Assign higher loss weight to minority class in training	Simple, no data augmentation needed
Threshold tuning	Move decision threshold from 0.5 to favour minority class recall	When precision-recall trade-off is key
Ensemble (BalancedBagging)	Each bag is balanced by under-sampling majority	Large imbalance with tree-based models

Module 7: Model Evaluation and Validation

A model that performs well on training data but poorly on new data is useless. Rigorous evaluation is the scientific heart of machine learning — without it, we have no basis for comparing algorithms, selecting hyperparameters, or claiming that our system works.

7.1 Classification Metrics

7.1.1 The Confusion Matrix

For binary classification with classes Positive (P) and Negative (N), the confusion matrix summarises all possible prediction outcomes:

	Predicted Positive	Predicted Negative
Actual Positive	True Positive (TP)	False Negative (FN) — Type II Error
Actual Negative	False Positive (FP) — Type I Error	True Negative (TN)

7.1.2 Performance Metrics Derived from the Confusion Matrix

$$\begin{aligned} \text{Accuracy} &= (TP + TN) / (TP + FP + FN + TN) \\ \text{Precision} &= TP / (TP + FP) \quad [\text{also called Positive Predictive Value}] \\ \text{Recall} &= TP / (TP + FN) \quad [\text{also called Sensitivity or True Positive Rate}] \\ \text{F1 Score} &= 2 \times (\text{Precision} \times \text{Recall}) / (\text{Precision} + \text{Recall}) \quad [\text{harmonic mean}] \\ \text{Specificity} &= TN / (TN + FP) \quad [\text{True Negative Rate}] \\ \text{Matthews Correlation Coefficient (MCC)} &= (TP \cdot TN - FP \cdot FN) / \sqrt{(TP+FP)(TP+FN)(TN+FP)(TN+FN)} \end{aligned}$$

 **Note:** Accuracy is misleading on imbalanced datasets. If 95% of samples are negative, a classifier that always predicts 'negative' achieves 95% accuracy while having 0% recall. Use F1 score, MCC, or balanced accuracy for imbalanced problems.

7.1.3 ROC Curve and AUC

The Receiver Operating Characteristic (ROC) curve plots the True Positive Rate (Recall) against the False Positive Rate (FPR = FP/(FP+TN)) as the classification threshold varies from 0 to 1. Each threshold gives one point on the curve.

AUC-ROC = Area Under the ROC Curve [probability that the model ranks a random positive above a random negative]

AUC-ROC interpretation:

- AUC = 1.0: perfect classifier
- AUC = 0.5: random classifier (diagonal line — no discrimination ability)
- AUC < 0.5: worse than random (predictions are inverted)
- AUC > 0.9: excellent; 0.8–0.9: good; 0.7–0.8: acceptable; < 0.7: poor

The Precision-Recall (PR) curve is more informative than ROC for highly imbalanced datasets, where the negative class dominates the FPR calculation.

7.2 Validation Strategies

7.2.1 Holdout Validation

The simplest approach: randomly split data into training (typically 70–80%) and test (20–30%) sets. Train on training set, evaluate on test set. The test set must never be used during model selection or hyperparameter tuning — only for final evaluation.

 **Note:** A common mistake is to tune hyperparameters on the test set, then report test set performance as an honest estimate of generalisation. This leads to optimistic, overfitted results. Use a separate validation set or cross-validation for hyperparameter tuning.

7.2.2 k-Fold Cross-Validation

k-fold cross-validation provides a less noisy estimate of generalisation performance by using all available data for both training and validation:

38. Randomly shuffle the dataset and divide into k equal folds (typically k = 5 or k = 10)
39. For each fold i = 1 to k: train on all folds except fold i, evaluate on fold i
40. Report mean and standard deviation of performance across all k evaluations

CV Score = $(1/k) \sum_i \text{Score}(\text{model trained on } D - D_i, \text{ evaluated on } D_i)$

Stratified k-fold maintains the class distribution in each fold — essential for imbalanced problems. Leave-One-Out CV (LOOCV) uses $k = m$ (each training example is held out once) — low bias but high variance and computationally expensive for large m .

7.3 Bias-Variance Trade-Off

The generalisation error of any model can be decomposed into three components:

$$\text{Expected Error} = \text{Bias}^2 + \text{Variance} + \text{Irreducible Noise}$$

Bias: The error introduced by the model's assumptions. High-bias models (linear regression on non-linear data) underfit — they cannot capture the true complexity. Reducing bias requires more complex models.

Variance: The sensitivity of the model to fluctuations in the training data. High-variance models (deep decision trees, $k=1$ k-NN) overfit — they memorise training data but fail on new data. Reducing variance requires regularisation, more data, or simpler models.

Irreducible Noise: The inherent noise in the data that no model can eliminate — the Bayes error rate.

Characteristic	Underfitting (High Bias)	Overfitting (High Variance)
Training error	High	Very low
Validation/test error	High	Much higher than training error
Decision boundary	Too simple (straight line on curved data)	Too complex (wiggly, memorises noise)
Fix	More complex model, more features	Regularisation, more data, fewer features

7.4 Hyperparameter Tuning

Hyperparameter: A parameter set before training that controls the learning process (e.g., k in k-NN, C in SVM, depth in decision tree, learning rate in neural network). Distinguished from model parameters, which are learned from data.

7.4.1 Grid Search

Exhaustively evaluates all combinations of hyperparameter values from a specified grid. Guaranteed to find the best combination within the grid, but scales exponentially with the number of hyperparameters.

```
from sklearn.model_selection import GridSearchCV
param_grid = {'C': [0.1, 1, 10], 'gamma': [0.001, 0.01, 0.1], 'kernel':
['rbf', 'linear']}
gs = GridSearchCV(SVC(), param_grid, cv=5, scoring='f1', n_jobs=-1)
gs.fit(X_train, y_train)
print(gs.best_params_, gs.best_score_)
```

7.4.2 Random Search

Samples hyperparameter combinations randomly from specified distributions. Empirically, random search is more efficient than grid search when only a few hyperparameters strongly affect performance — it explores more diverse combinations.

7.4.3 Bayesian Optimisation

Models the objective function (validation performance) with a probabilistic surrogate model (typically a Gaussian Process or Tree Parzen Estimator). Uses the surrogate to select the next hyperparameter configuration most likely to improve performance. Significantly more efficient than grid or random search for expensive-to-evaluate models. Libraries: Optuna, Hyperopt, Scikit-Optimize.

Module 8: Text Mining, Web Mining, and Stream Mining

Real-world data mining increasingly involves unstructured text, hyperlinked web pages, and continuous data streams. This module covers the specialised techniques required for each domain.

8.1 Text Mining and Natural Language Processing (NLP)

Text mining extracts structured information from unstructured text. The central challenge is converting variable-length sequences of words into fixed-dimensional numerical feature vectors that machine learning algorithms can process.

8.1.1 Text Pre-processing Pipeline

41. Tokenisation: split text into tokens (words, sub-words, characters)
42. Lowercasing: convert all text to lowercase (optional — preserves proper nouns in named entity recognition)
43. Stop word removal: remove common words ('the', 'a', 'is') that carry little discriminative information
44. Stemming / Lemmatisation: reduce words to their root form (running → run, better → good)
45. Noise removal: strip HTML tags, URLs, punctuation, special characters

8.1.2 Text Representation

Bag of Words (BoW): Represents a document as a vector of word counts, ignoring word order and grammar. Simple but sparse (vocabulary size can be 100,000+). Example: 'the cat sat on the mat' → {cat:1, mat:1, sat:1, the:2, on:1}

TF-IDF (Term Frequency-Inverse Document Frequency): Weights each word by how often it appears in the document (TF) divided by how commonly it appears across all documents (IDF). Penalises common words, rewards distinctive ones.

$$\text{TF-IDF}(t, d) = \text{TF}(t, d) \times \log(N / \text{df}(t))$$

Where N is total documents and $\text{df}(t)$ is the number of documents containing term t .

Word2Vec: Learns dense word embeddings (typically 100–300 dimensions) such that semantically similar words have similar vectors. Trained using CBOW (predict word from context) or Skip-gram (predict context from word) on large corpora.

GloVe (Global Vectors): Word embeddings trained on word co-occurrence statistics from a global corpus. Captures semantic relationships: $\text{vec}(\text{'king'}) - \text{vec}(\text{'man'}) + \text{vec}(\text{'woman'}) \approx \text{vec}(\text{'queen'})$

Transformer Embeddings (BERT, GPT): Contextual embeddings where the vector for each word depends on its context in the sentence. Far more expressive than static embeddings. BERT's embeddings are the standard for most NLP tasks.

8.1.3 Text Classification Tasks

Task	Description	Example Applications
Sentiment Analysis	Classify text as positive/negative/neutral	Product reviews, social media monitoring, brand analysis
Topic Modelling (LDA)	Discover latent topics in a corpus (unsupervised)	News categorisation, document organisation, trend discovery
Named Entity Recognition	Label spans of text as entities (person, org, location)	Information extraction, knowledge graph construction
Text Classification	Assign a text to one or more predefined categories	Spam filtering, news category, intent classification
Question Answering	Extract or generate answers to natural language questions	Customer support bots, search engines, virtual assistants

8.2 Web Mining

Web mining applies data mining techniques to the World Wide Web, exploiting its unique structure: billions of pages connected by hyperlinks, containing text, images, and user interaction logs.

Web Content Mining: Mining the content of web pages (text, images, video). Includes web scraping, sentiment analysis of reviews, extracting structured data from HTML.

Web Structure Mining: Analysing the hyperlink structure of the web. PageRank and HITS are landmark algorithms that use link structure to rank page authority.

Web Usage Mining (Clickstream Analysis): Analysing user behaviour logs (page views, session paths, click sequences) to understand user navigation patterns, personalise recommendations, and improve site design.

8.2.1 PageRank Algorithm

Google's PageRank models the web as a directed graph where each page is a node and hyperlinks are directed edges. A page's rank is the probability that a random surfer — following links at random, with a small probability d of jumping to a random page — lands on it.

$$PR(A) = (1-d)/N + d \cdot \sum_{B \rightarrow A} PR(B) / \text{OutDegree}(B)$$

Where $d \approx 0.85$ is the damping factor and N is the total number of pages. PageRank is computed iteratively until convergence. Pages linked to by many high-PageRank pages receive high PageRank themselves.

8.3 Stream Mining

Data streams are continuous, potentially infinite sequences of data arriving at high speed, where storing the entire stream is infeasible. Stream mining algorithms must process each element in a single pass, using bounded memory, and adapt to concept drift (changes in the underlying data distribution over time).

8.3.1 Challenges of Stream Mining

- Single-pass constraint: each data item can be examined at most once (or a small constant number of times)
- Memory constraint: the working memory is far smaller than the total data stream
- Speed constraint: processing must keep up with the arrival rate
- Concept drift: the data distribution may change over time, making models trained on old data stale

8.3.2 Key Algorithms

Sliding Window: Maintains the last W items in memory; statistics are computed over this window. Simple and adaptive but requires W items of memory.

ADWIN (Adaptive Windowing): Dynamically adjusts the window size to detect concept drift: if the mean of two sub-windows differs significantly (by more than a threshold), the older sub-window is discarded.

Hoeffding Trees (VFDT): Decision trees for data streams: use the Hoeffding bound to determine how many samples are needed to make a split decision with high confidence, avoiding storing all data.

Reservoir Sampling: Maintains a representative random sample of size R from a stream of unknown length, with each item having equal probability R/n of being in the sample at any time.

Module 9: Ethics, Fairness, and Responsible Data Mining

Pattern recognition and data mining systems make consequential decisions: who gets a loan, who is flagged as a security threat, who is shown which medical information. These decisions can perpetuate or amplify social inequalities if built without careful ethical consideration.

9.1 Bias in Machine Learning

9.1.1 Sources of Bias

Historical Bias: The training data reflects historical inequalities (e.g., past hiring decisions that favoured men). A model trained on this data learns to replicate discrimination.

Representation Bias: Certain groups are under-represented in the training data (e.g., face recognition trained mostly on light-skinned faces performs poorly on dark-skinned faces).

Measurement Bias: Features or labels are measured differently across groups (e.g., arrest rates as a proxy for criminality, when arrest rates are themselves affected by racially biased policing).

Feedback Loop Bias: A model's predictions affect future training data (e.g., a predictive policing model sends more patrols to already over-policed areas, generating more arrests there, which reinforces the model's predictions).

9.2 Fairness Metrics

Multiple competing mathematical definitions of fairness exist. They are generally mutually incompatible — you cannot satisfy all of them simultaneously when base rates differ across groups.

Fairness Criterion	Definition	Example
Demographic Parity	Equal proportion of positive predictions across groups	Loan approval rate equal for all racial groups
Equal Opportunity	Equal recall (TPR) across groups	Equal re-offending detection rate across races
Equalised Odds	Equal TPR and FPR across groups	Equal true and false positive rates in medical screening
Individual Fairness	Similar individuals receive	Two candidates with

	similar predictions	equal qualifications get equal scores
Calibration	Predicted probabilities match actual frequencies per group	A 70% predicted risk means 70% actual risk for all groups

 **Note:** The COMPAS case study: a recidivism prediction algorithm used in US courts was found to have equal accuracy across racial groups but higher false positive rates for Black defendants (incorrectly flagging low-risk individuals as high-risk) and higher false negative rates for White defendants. This satisfies calibration but violates equal opportunity — illustrating that different stakeholders may prioritise different fairness criteria.

9.3 Privacy in Data Mining

- **Differential Privacy:** a mathematical framework that adds calibrated noise to query results or training processes so that the presence or absence of any individual's data has negligible effect on the output. Used by Apple, Google, US Census Bureau.
- **k-Anonymity:** ensures that each individual in a published dataset is indistinguishable from at least $k-1$ others with respect to quasi-identifier attributes (age, postcode, gender).
- **Federated Learning:** train models on decentralised data (e.g., on users' devices) without collecting the data centrally. Only model gradients — not raw data — are shared and aggregated.
- **Data minimisation:** collect only the minimum personal data necessary for the stated purpose — a GDPR requirement and sound engineering practice.

9.4 Explainability and Interpretability

As machine learning models are deployed in high-stakes decisions, the ability to explain why a model made a particular prediction becomes legally and ethically essential (GDPR grants individuals the 'right to explanation' for automated decisions).

Inherently Interpretable Models: Models whose decision logic can be directly inspected: decision trees (follow the path), linear models (inspect weights), rule-based systems.

LIME (Local Interpretable Model-Agnostic Explanations): Fits a simple interpretable model (linear regression) locally around a specific prediction point. Explains any black-box model's prediction for a single instance.

SHAP (SHapley Additive exPlanations): Assigns each feature a Shapley value — its average marginal contribution to the prediction across all possible feature subsets. Theoretically grounded, consistent, and applicable to any model. Industry standard for model explainability.

```
import shap
explainer = shap.TreeExplainer(rf_model)      # for tree models
shap_values = explainer.shap_values(X_test)
shap.summary_plot(shap_values, X_test)        # global feature importance
shap.waterfall_plot(explainer(X_test[0]))     # explain single prediction
```

Revision Guide and Examination Preparation

10.1 Key Concepts Summary by Module

Module 1 — Foundations

- Pattern recognition pipeline: 7 stages from data collection to deployment
- Feature types: continuous, ordinal, nominal, binary, text, image — and appropriate encodings
- Curse of Dimensionality: sparsity, distance concentration, exponential sample requirement
- Bayes' theorem: $\text{posterior} = \text{likelihood} \times \text{prior} / \text{evidence}$; optimal decision rule
- Types of learning: supervised, unsupervised, semi-supervised, reinforcement, self-supervised

Module 2 — Classical Classification

- k-NN: non-parametric, lazy, distance-based; choosing k; distance metrics; limitations at scale
- Naive Bayes: conditional independence assumption; Gaussian/Multinomial/Bernoulli variants; Laplace smoothing
- Decision Trees: entropy, information gain, Gini impurity; pruning strategies; CART vs ID3/C4.5
- SVM: maximal margin hyperplane; soft margin; kernel trick (RBF, polynomial, linear)
- Ensemble methods: bagging (Random Forest), boosting (AdaBoost, XGBoost), stacking

Module 3 — Neural Networks

- Neuron, activation functions: Sigmoid, ReLU, Softmax, GELU — when to use each
- Backpropagation: chain rule, forward pass, backward pass, weight update
- Regularisation: L2, Dropout, Batch Normalisation, Early Stopping
- CNNs: convolutional layer, pooling, ResNet skip connections, transfer learning
- RNNs / LSTMs: sequential processing, vanishing gradient, forget/input/output gates

Module 4 — Unsupervised Learning

- k-Means: Lloyd's algorithm, WCSS objective, elbow method, silhouette score; limitations
- Hierarchical clustering: agglomerative; single/complete/average/Ward linkage; dendrogram
- DBSCAN: core/border/noise points; ϵ and MinPts; arbitrary-shape clusters; no k needed
- PCA: covariance matrix, eigendecomposition, explained variance, scree plot, Eigenfaces
- t-SNE vs UMAP: visualisation vs preprocessing; local vs global structure preservation

Module 5 — Association Rules

- Support, Confidence, Lift — formulas and interpretation
- Apriori: antimonotone property, candidate generation, multiple database scans
- FP-Growth: FP-tree construction (2 scans), conditional pattern base, conditional FP-tree
- Rule evaluation: lift, conviction, leverage — filtering spurious rules

Module 6 — Feature Engineering

- Missing data: deletion, mean/median/mode imputation, MICE, indicator variable
- Categorical encoding: one-hot, ordinal, target encoding, embeddings
- Feature scaling: Min-Max, Z-score, Robust Scaler, Log transform
- Feature selection: filter (MI, Chi-squared), wrapper (RFE), embedded (Lasso, RF importance)
- Imbalanced datasets: SMOTE, class weights, threshold tuning

Module 7 — Model Evaluation

- Confusion matrix: TP, FP, FN, TN — accuracy, precision, recall, F1, MCC
- ROC curve and AUC interpretation; PR curve for imbalanced data
- k-fold cross-validation: stratified, LOOCV; train/val/test split disciplines
- Bias-variance trade-off: bias^2 , variance, irreducible noise; underfitting vs overfitting

- Hyperparameter tuning: grid search, random search, Bayesian optimisation (Optuna)

Module 8 — Text, Web, and Stream Mining

- Text pre-processing pipeline: tokenisation, stop words, stemming, lemmatisation
- Text representations: BoW, TF-IDF, Word2Vec, BERT contextual embeddings
- Web mining: content, structure (PageRank), usage (clickstream)
- Stream mining challenges: single-pass, memory, speed, concept drift
- Stream algorithms: sliding window, ADWIN, Hoeffding trees, reservoir sampling

Module 9 — Ethics and Fairness

- Bias sources: historical, representation, measurement, feedback loop
- Fairness metrics: demographic parity, equal opportunity, equalised odds — and their incompatibility
- Privacy: differential privacy, k-anonymity, federated learning
- Explainability: LIME (local linear surrogate), SHAP (Shapley values) — interpret any model

10.2 Sample Examination Questions with Guidance

Question 1 (12 marks)

Explain the Bayes optimal classifier. Derive the decision rule for a two-class problem and explain what the Bayes error rate represents. Why can this classifier rarely be implemented in practice?

Guidance:

State Bayes' theorem. Show that the optimal rule assigns x to class $c^* = \operatorname{argmax}_c P(c|x) = \operatorname{argmax}_c P(x|c) \cdot P(c)$ ($P(x)$ cancels). The Bayes error rate is the minimum achievable error rate — it equals the integral of $\min_c P(c|x)$ over all x . In practice, we cannot implement it because $P(x|c)$ is unknown — we only have finite training samples from which we must estimate it. Estimation error means all practical classifiers have error rates above the Bayes rate.

Question 2 (10 marks)

Compare the k-Means and DBSCAN clustering algorithms. For each of the following datasets, explain which algorithm is more appropriate and why: (a) 5 spherical clusters of similar density; (b) a dataset containing two crescent-shaped clusters; (c) a dataset with many outliers (anomaly detection task).

Guidance:

k-Means: requires k , spherical clusters, sensitive to outliers, minimises WCSS, $O(n \cdot k \cdot l)$ per iteration. DBSCAN: no k required, arbitrary shapes, identifies noise, requires ϵ and MinPts, $O(n \log n)$. (a) k-Means is ideal — spherical, similar density matches its assumptions; DBSCAN would also work but requires careful ϵ tuning. (b) DBSCAN is clearly superior — the crescent shape is non-convex and k-Means would split the crescents incorrectly. (c) DBSCAN again — it explicitly marks points as noise (outliers); k-Means has no noise concept and forces every point into a cluster.

Question 3 (15 marks)

You are building a fraud detection system for a bank. The dataset has 1 million transactions, of which 0.1% are fraudulent. (a) Why is accuracy an inappropriate evaluation metric here? (b) Which metric(s) would you use and why? (c) Describe THREE techniques to address the class imbalance. (d) A colleague claims their Random Forest achieves 99.95% accuracy — is this impressive? Explain.

Guidance:

(a) Accuracy is misleading: a classifier predicting 'not fraud' for all transactions gets 99.9% accuracy but catches zero frauds. (b) Use: Recall/Sensitivity (we care most about catching fraud — false negatives are costly), Precision (avoiding false alerts — false positives waste investigator time), F1 or F-beta (if recall is more important, use F-beta with $\beta > 1$), AUC-PR (better than AUC-ROC for imbalanced data), MCC (balanced single metric). (c) SMOTE (synthetic oversampling of fraud class), class weights (set `class_weight='balanced'` in sklearn), threshold tuning (move decision threshold from 0.5 to 0.2 to favour recall). (d) Not impressive — a naive classifier achieves 99.9% accuracy. The colleague's model at 99.95% only catches marginally more fraud than doing nothing. Evaluate with recall, precision, and F1 on the fraud class.

Question 4 (10 marks)

Explain the bias-variance trade-off. For each of the following scenarios, state whether the model is likely suffering from high bias or high variance, and suggest one concrete technique to address it: (a) A decision tree with `max_depth=None` has 100% training accuracy but 62% test accuracy; (b) A linear SVM has 74% training accuracy and 72% test accuracy on a dataset with clear non-linear patterns.

Guidance:

Bias-variance decomposition: Expected Error = Bias² + Variance + Irreducible Noise. (a) High variance (overfitting): training error ≈ 0 , test error $\gg$ training error. Fix: pruning (set `max_depth`, `min_samples_leaf`), or use Random Forest to average out the variance. (b) High bias (underfitting): training and test error both poor, small gap between them. The linear kernel cannot capture non-linear patterns. Fix: switch to RBF kernel (or polynomial kernel) to map to higher-dimensional feature space where the data may be linearly separable.

Question 5 (8 marks)

Explain the FP-Growth algorithm for mining frequent itemsets. What is the key advantage of FP-Growth over Apriori, and what is the role of the antimonotone property in Apriori that FP-Growth exploits differently?

Guidance:

Apriori: (1) requires multiple database scans — one per itemset length level; (2) generates large numbers of candidate itemsets that must be counted. The antimonotone property (if X is infrequent, all supersets of X are infrequent) is used to prune the candidate set. FP-Growth: (1) requires exactly 2 database scans — one to find frequent items, one to build the FP-tree; (2) compresses the database into a compact tree structure in memory; (3) mines directly from the tree by building conditional FP-trees recursively, exploiting antimonotonicity implicitly (items below minsup are never added to the tree). Key advantage: no candidate generation and only 2 scans make FP-Growth orders of magnitude faster on dense datasets.

10.3 Glossary of Key Terms

Term	Definition
AUC-ROC	Area Under the Receiver Operating Characteristic Curve — probability of correctly ranking a positive above a

	random negative
Bagging	Ensemble method: train models on bootstrap samples in parallel; combine by voting/averaging
Bayes Error Rate	The minimum achievable classification error rate — a lower bound for all classifiers
Boosting	Ensemble method: train models sequentially, focusing each on the previous model's mistakes
Confusion Matrix	Table showing TP, FP, FN, TN for a classifier's predictions
Cross-validation	Model evaluation by training and testing on multiple data splits to reduce evaluation variance
DBSCAN	Density-Based Spatial Clustering of Applications with Noise — finds arbitrary-shape clusters
Dropout	Regularisation: randomly zero out neurons during training to prevent co-adaptation
Entropy	Information-theoretic measure of impurity/disorder in a set: $H = -\sum p \log_2 p$
FP-Growth	Frequent Pattern Growth — mines association rules using a compressed FP-tree (2 DB scans)
Gini Impurity	Decision tree splitting criterion: $1 - \sum p_i^2$ (probability of misclassifying random element)
Gradient Boosting	Sequential ensemble: each tree fits the residuals of the current ensemble
Information Gain	Reduction in entropy after a dataset is split on a feature — used by ID3/C4.5
k-NN	k-Nearest Neighbours — classifies by majority vote of k closest training examples
Kernel Trick	Implicitly maps features to high-dimensional space via kernel function $K(x,z) = \phi(x) \cdot \phi(z)$
LIME	Local Interpretable Model-Agnostic Explanations — fits local linear model to explain any prediction
LSTM	Long Short-Term Memory — RNN variant with forget/input/output gates for long-range dependencies
Naive Bayes	Probabilistic classifier assuming feature conditional independence given the class
Overfitting	Model learns training data noise; high training accuracy but poor generalisation
PCA	Principal Component Analysis — linear dimensionality

	reduction by projecting onto directions of maximum variance
Random Forest	Bagging ensemble of decision trees with random feature selection at each split
SHAP	SHapley Additive exPlanations — assigns each feature its average marginal contribution
SMOTE	Synthetic Minority Over-sampling Technique — generates synthetic minority class samples
SVM	Support Vector Machine — finds the maximal-margin hyperplane separating classes
t-SNE	t-Distributed Stochastic Neighbour Embedding — non-linear 2D/3D visualisation technique
Transfer Learning	Re-using a model trained on one task as the starting point for a related task
Underfitting	Model is too simple to capture data patterns; poor training and test performance
Vanishing Gradient	Gradients shrink exponentially during backpropagation through deep networks; solved by ReLU, LSTM, ResNets

10.4 Recommended Reading and Resources

Primary Textbooks

- Bishop, C.M. (2006). Pattern Recognition and Machine Learning. Springer — the definitive graduate-level reference; rigorous probabilistic treatment
- Mitchell, T. (1997). Machine Learning. McGraw-Hill — accessible introduction; excellent chapters on decision trees and Bayesian learning
- Goodfellow, I., Bengio, Y., & Courville, A. (2016). Deep Learning. MIT Press — comprehensive deep learning reference (free online at deeplearningbook.org)
- Han, J., Kamber, M., & Pei, J. (2011). Data Mining: Concepts and Techniques (3rd ed.). Morgan Kaufmann — covers association rules, clustering, and classification comprehensively
- Hastie, T., Tibshirani, R., & Friedman, J. (2009). The Elements of Statistical Learning (2nd ed.). Springer — rigorous mathematical treatment of all classical methods (free PDF online)

Online Resources

- [scikit-learn documentation: scikit-learn.org](https://scikit-learn.org) — comprehensive Python ML library reference with examples
- [fast.ai Practical Deep Learning for Coders: fast.ai](https://fast.ai) — hands-on deep learning course with PyTorch
- [Kaggle Learn: kaggle.com/learn](https://kaggle.com/learn) — free micro-courses on ML, feature engineering, and deep learning
- [Distill.pub](https://distill.pub) — interactive, visually rich explanations of ML concepts including attention mechanisms and t-SNE
- [Papers With Code: paperswithcode.com](https://paperswithcode.com) — state-of-the-art results with linked implementations

Hands-On Datasets for Practice

- [UCI ML Repository \(archive.ics.uci.edu\)](https://archive.ics.uci.edu): 500+ benchmark datasets including Iris, Wine, Heart Disease
- [Kaggle Competitions: kaggle.com/competitions](https://kaggle.com/competitions) — real-world datasets with leaderboards
- [OpenML \(openml.org\)](https://openml.org): curated benchmark suites for systematic algorithm comparison
- MNIST, CIFAR-10, ImageNet: standard image classification benchmarks built into most frameworks

— End of CSD622 Lecture Notes —

Dr A.J Fofanah · CSD622 Pattern Recognition and Data Mining